INDEX PAGE

Sr. No	Contents	Page No.	Grade	Signature
1	Identifying the Requirements from Problem Statements			
2	Estimation of Project Matrices			
3	Modeling UML Use Case Diagrams and Capturing Use Case Scenarios			
4	E-R Modeling from the Problem Statements			
5	Identifying Domain Classes from the Problem Statements			
6	Statechart and Activity Modelling			
7	Modeling UML Class Diagrams and Sequence Diagrams			
8	Modeling Data Flow Diagrams			
9	Estimation of Test Coverage Metrices and Structural Complexity			
10	Designing Test Suites			

EXP: 1	AIM:- In this experiment we will learn how to identify functional and non-functional requirements from a given problem statement. Functional and non-functional requirements are the primary components of a Software Requirements Specification.

INTRODUCTION:

Requirements identification is the first step of any software development project. Until the requirements of a client have been clearly identified, and verified, no other task (design, coding, testing) could begin. Usually business analysts having domain knowledge on the subject matter discuss with clients and decide what features
are to be implemented.

PRETEST

When is feasibility study done?
- a: After requirements specifications have been finalized
- b: During the period when requirements specifications are prepared
- c: Before the final requirements specifications are done
- d: Could be done at eny time

A good requirement specification is one, which is
- a: Consistent
- b: Complete
- c: Unambiguous
- d: All of the above

Requirement specification is done
- a: After requirements are determined
- b: Before requirements are determined
- c: Simultaneously with requirements determination
- d: Independent of requirements determination

Submit Quiz
Score: 3 out of 3

POSTTEST

Functional requirements of a system are related to
- a: Using the system (by users) to get some meaningful work done
- b: How the system functions under different constraints
- c: Whether they adhere to the organization policies

SRS refers to
- a: Software Requirements Specification
- b: System Resources Statement
- c: Statement of Reliability of System
- d: Standard Requirements Statement

The main objective behind preparing a SRS is to
- a: Let client and developers agree that they understand each other
- b: Formally note down the requirements
- c: Estimate the cost of development
- d: To judge whether the project could be undertaken

Submit Quiz
Score: 3 out of 3

SIMULATION :

We show here how to extract functional requirements when a problem statement is given. The case under study is a online voting system.

Internet has led to discussion of e-democracy and online voting. Many peoples think that the internet could replace representativedemocracy, enabling everyone to vote on everything and anything by online voting . Online voting could reduce cost and make voting more convenient. This type of voting can be done for e-democracy, or it may be used for finalizing a solution, if many alternatives are present. Online voting make's use of authentication, hence it needs security, and the system must be able to address obtaining, marking, delivering and counting ballots via computer. Advantage of online voting is it could increase voter turnout because of convenience, and it helps to reduce fraud voting.

Internet has led to discussion of e-democracy and online voting. Many peoples think that the internet could replace representativedemocracy, enabling everyone to vote on everything and anything by online voting. Online voting could reduce cost and make voting more convenient. This type of voting can be done for e-democracy, or it may be used for finalizing a solution, if many alternatives are present. Online voting make's use of authentication, hence it needs security, and the system must be able to address obtaining, marking, delivering and counting ballots via computer. Advantage of online voting is it could increase voter turnout because of convenience, and it helps to reduce fraud voting.

Vote: A user can cast his vote in favour of a particular candidate.

PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. From the given problem statement, try to figure out if there's any inconsistency with the requirement specification
2. Also, try to determine what are the functional and non-functional requirements are
3. Select the check boxes accordingly, and then click on the 'Submit' button

CONCLUSION

In this experiment, we successfully learned how to identify and differentiate functional and non-functional requirements from a given problem statement. Functional requirements describe what the system should do, while non-functional requirements describe how the system should perform in terms of quality, security, reliability, usability, and performance. This experiment helped in understanding the importance of clearly defining requirements for preparing an effective Software Requirements Specification (SRS) and for developing a software system that meets user and business needs.

EXP: 2	**AIM:- In this experiment, we will learn how to estimate cost, effort and duration for a software project, and then select one solution approach which will be found suitable to fulfill the organizational goal.**

INTRODUCTION

After gathering the entire requirements specific to software project usually we need to think about different solution strategy for the project. Expert business analysts are analyzing their benefits and as well as their shortcomings by means of cost, time and resources require to develop it.

PRETEST

According to the COCOMO model, a project can be categorized into
- a: 3 types
- b: 5 types
- c: 5 types
- d: No such categorization

In Intermediate COCOMO model, Effort Adjustment Factor (EAF) is derived from the effort multipliers by
- a: Adding them
- b: Multiplying them
- c: Taking their weighted average
- d: Considering their maximum

Project metrics are estimated during which phase?
- a: Feasibility study
- b: Planning
- c: Design
- d: Development

Submit Quiz

Score: 3 out of 3

POSTTEST

According to Halsetad's metrics, program length is given by the
- a: Sum of total number of operators and operands
- b: Sum of number of unique operators and operands
- c: Total number of operators
- d: Total number of operands

Complete COCOMO considers a software as a
- a: Homogeneous system
- b: Heterogeneous system

Consider you are developing a web application, which would make use of a lot of web servic
it be wise to make estimates for this project using COCOMO?
- a: Yes, of course
- b: Not at all

Submit Quiz

Score: 3 out of 3

SIMULATION

Using Basic COCOMO model to estimate project parameters

Use the simulator on the right hand side to understand how project type and size affects the different parameters estimated. *Quick glance at the formulae:*

- ***Effort:*** $a * (Size)^b$ *person-month*
- ***Tdev :*** $2.5 * (Effort)^c$ *month*
- **No of Developers:** Effort/Tdev

Drag the slider to change the project size. Note: select the nearest discrete value corresponding to the actual size.

Project Type	a	b	c
Organic	2.4	1.05	0.38
Project size (in KLOC)	18		
Effort (in PM)	49.92		
Tdev (in month)	11.05		
# of developers	5		

As evident from the simulation parameters, size of a semi-detached project is larger than that of an organic project, and size of an embedded project is larger than that of a semi-detached, and thereby affecting factors like effort and development time.

PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. Select exercise number from the drop down.
2. The 'Exercises 1' is to calculate COCOMO model. Select Project Type from the dropdown. Then enter numeric value in project size and Effort Adjustment Factor (EAF) input field. Click on "Calculate" button to view the result.
3. The 'Exercises 2' is to identify operators and operands from a given problem statement. Tick on the operators and operands and click on "Check" button.
4. The 'Exercises 3' is to calculate total no. of operators and operands from a given problem statement. click on "Calculate" button to view result.
5. To view the solution of the exercises, click on the 'Submit' button, and then on the 'View Solution' button.

CONCLUSION

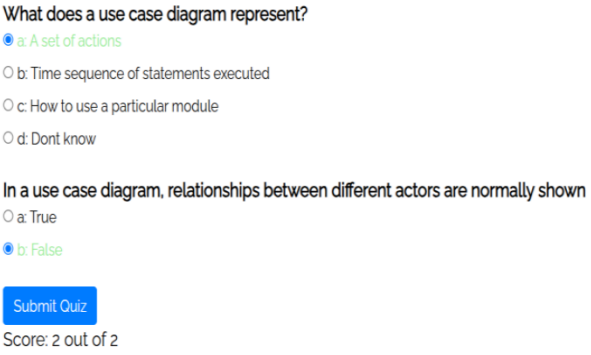
In this experiment, we successfully learned how to estimate the cost, effort, and duration of a software project using appropriate estimation techniques. We also compared different solution approaches based on project requirements, resources, time, and organizational goals. This experiment helped us understand the importance of accurate project estimation and proper selection of a suitable solution approach for the successful and efficient development of a software project.

EXP: 3	AIM:- In this experiment, we will learn how use cases and actors can be captured and how different use cases are related in a system.

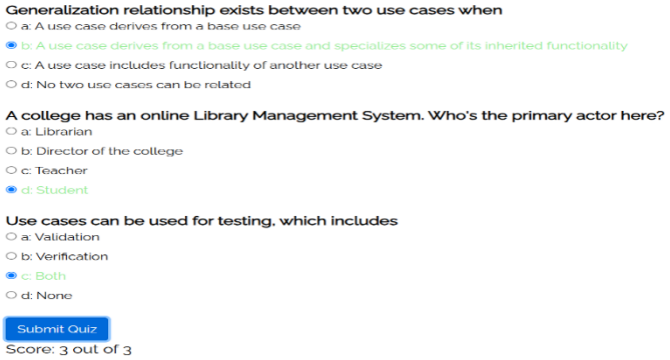
INTRODUCTION

Use case diagram is a platform that can provide a common understanding for the end-users, developers and the domain experts. It is used to capture the basic functionality i.e. use cases, and the users of those available functionality, i.e. actors, from a given problem statement.

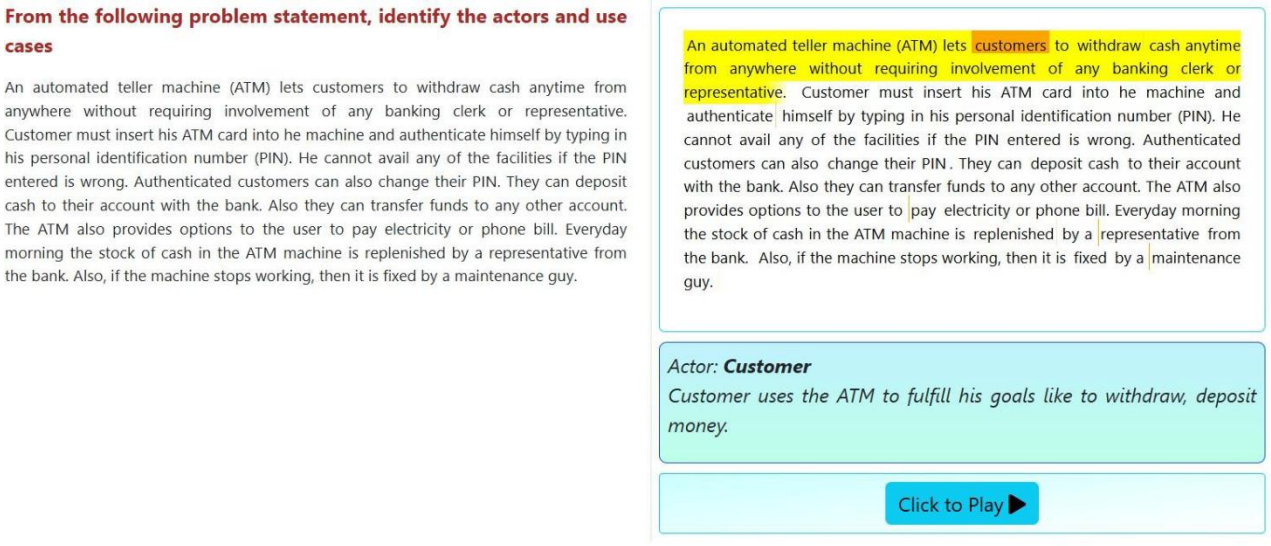
PRETEST



POSTTEST



SIMULATION



PROCEDURE

Steps for conducting the experiment

General Instructions
Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions
Following are the instructions specifically for this experiment:

1. Identify an actor, and write its name in the textbox of 'Table #1'. Click the adjacent 'Add' button to add this actor. Repeat this for all the possible actors.
2. Identify a use case, and write its name in the textbox of 'Table #2'. Click the adjacent 'Add' button to add this use case. Repeat this for all the possible use cases.
3. If you want to delete any actor or use case, go to the 'Table #4', remove the actor or use case.
4. 'Table #3' lets you define relationships between any two components. Select the first actor/use case from the first dropdown list, the second from the third drop down list. Select one relationship from the second dropdown list. If you want to put any label to the relation, write the text for the label to the adjacent textbox. Click on the 'Add' button at the right side to add this relation. Repeat this for all the possible relations.
5. Relationships so defined will be displayed in the 'Table #5'. Here, you have the option to remove a wrongly defined relationship.
6. Click on the 'Draw' button to draw the Use Case diagram. The Use Case diagram will be displayed when all the actors, use cases, or relationships are added in the table.
7. To view a sample solution for the exercise, click on the 'Submit' button, and then on the 'View Solution' button.

CONCLUSION

In this experiment, we successfully learned how to identify and represent actors and use cases in a software system. We also understood how different use cases can be related using relationships such as include, extend, and generalization. This experiment helped us understand how Use Case Diagrams represent the interaction between users (actors) and the system, making system requirements easier to analyze, communicate, and document.

EXP: 4	AIM:- In this experiment we will learn how to find the entities, its attributes and how the relationships between the entities can be established for a system.

INTRODUCTION

Developing databases is a very important task to develop a system. Before going to form exact database tables and establishing relationships between them, we conceptually or logically can model our database using ER diagrams.

PRETEST

In ER model an entity is usually related to another entity. The relationship is indicated b

- a: Rectangle
- b: Ellipse
- c: Triangle
- d: Diamond

Entity Relationship model is used to represent the database at a

- a: Logical level
- b: Physical level

Submit Quiz

Score: 2 out of 2

POSTTEST

An entity set could be related to itself

- a: True
- b: False

A prime attributes is indicated by

- a: Underline inside an ellipse
- b: Double ellipse
- c: Dotted ellipse
- d: Dotted rectangle

Students study in schools -- this is a

- a: 1:1 mapping
- b: 1:N mapping
- c: M:1 mapping
- d: M:N mapping

Submit Quiz

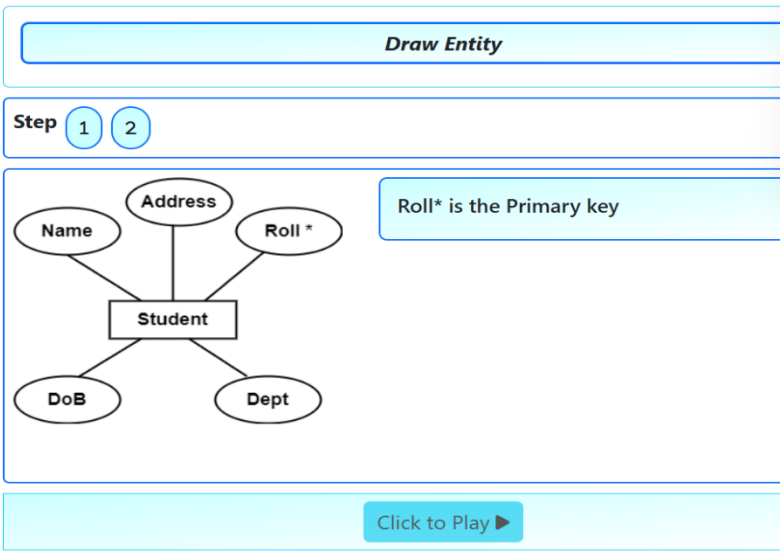
Score: 3 out of 3

SIMULATION

Draw an ER diagram for a School Management System based on the following information.

You have been asked to implement a database for a school management system (SMS). This primarily consists of maintaining students' information like name, address, date of birth, roll number, deparment, and so on. Details about the school to be stored includes school's name, location. Although it is unlikely that there would be two schools at the same place with same name, but our SMS would like to accomodate this possibility.

Faculty members works in the school. They teaches the students. A faculty member normally teaches multiple students at a time. Also, he can teach multiple courses to the students.



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. Identify an entity, and write its name in the textbox of Table #1. 'Tick' the adjacent checkbox in case it is a weak entity. Click the adjacent 'Add' button to add this entity. Repeat this for all the possible entities.
2. From Table #2, select one entity and add its possible attributes. If the attribute is a primary key or a member of composite primary key for the entity, select the check box under 'Primary key' heading.
3. If you want to delete/modify any attribute for an entity, go to Table #4, remove the attribute, and then add the correct one for the selected entity.
4. Table #3 lets you define relationships between any two entities along with the relationship cardinality. Select the first entity from the first dropdown list, the second from the second dropdown list, write in the relationship name in the text box, select the cardinality from the third drop down list, and click on the 'Add' button at the right side.
5. Relationships so defined will be displayed in Table #5. Here, you have the option to remove a wrongly defined relationship.
6. Click on the 'Draw' button to draw the ER diagram. Click on this button when all the entities, attributes, or relationships are added. The ER diagram will be displayed at the bottom of the page.
7. To view a sample solution for the exercise, click on the 'Submit' button, and then on the 'View Solution' button

CONCLUSION

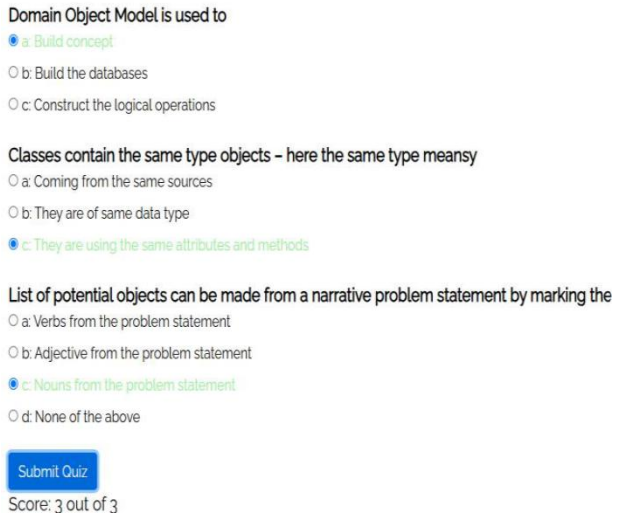
In this experiment, we successfully learned how to identify entities, their attributes, and the relationships between different entities in a given system. We also understood how to represent these components using an Entity-Relationship (ER) model and identify different types of relationships such as 1:1, 1:N, and M:N. This experiment helped us understand the basic structure of a database and how ER diagrams can be used to design and organize data effectively before implementing the database.

EXP: 5	AIM:- In this experiment we will learn how to identify the classes from a given problem statement.

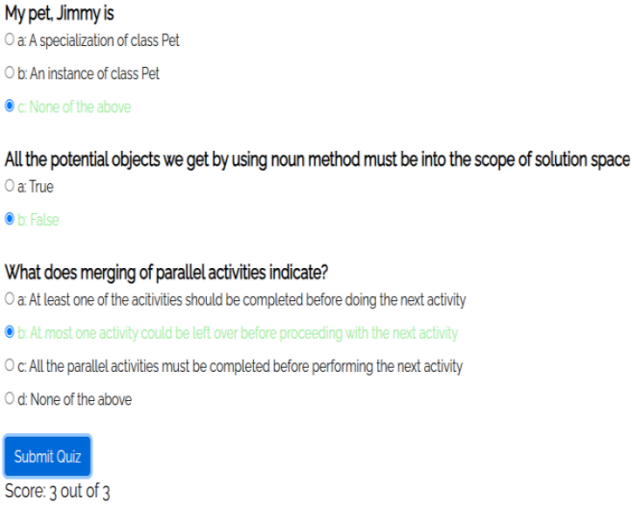
INTRODUCTION

Same types of objects are typically implemented by class in object oriented programming. As the structural unit of the system can be represented through the classes, so, it is very important to identify the classes before start implementing all the logical flows of the system.

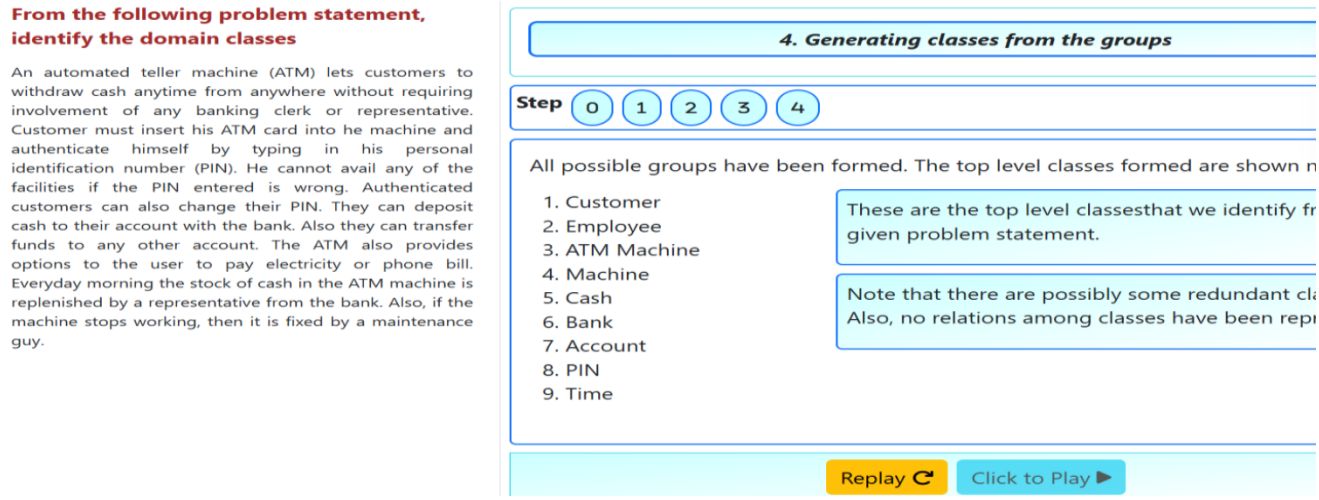
PRETEST



POSTTEST



SIMULATION



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. Identify a noun or noun phrase, and write it in the textbox of Table #1. Click the adjacent 'Add' button to add this noun or noun phrase. Repeat this until all the nouns and noun phrases are taken from the problem statement.
2. If you want to delete any noun phrase, you can do it from the list of Table #2.
3. In Table #2, we will categorize the nouns which are taken as Potential objects in our solution.
4. 'Tick' the adjacent checkbox of a potential object in the left-list of Table #2. Select a category from the middle drop down list. Click the adjacent 'Add' button to group the object under the selected category. Repeat this until all the potential objects are properly categorized.
5. All the potential objects under their selected category are shown in Table #3.
6. Table #4 lets you assign attributes to the objects. Select one object from the left drop down list. 'Tick' adjacent check boxes you want to assign to the selected object from the given attribute list. Click the adjacent 'Add' button to assign those attributes to the selected object .
7. You may add your own attribute from Table #5 to the attribute list of Table #4 if it is required.
8. If you want to remove an attribute what you have added, you can do it from the attribute list of Table #4.
9. Table #6 shows all the objects with its related attributes.
10. If you want to remove any attribute of an object shown in Table #6, you can do it easily from the same table and can re-insert the elements accordingly.
11. Table #7 is to define the classes based on common attributes it may have. Tick adjacent check boxes you want to have in a class from the left attribute list of Table #7. Give a class name in the middle textbox. Click the adjacent 'Add' button.
12. Table #8 shows all the classes based on some common attributes which are taken from Table #7.
13. If you want to remove any attribute of a class shown in Table #8, you can do it easily from the same table and can re-insert the elements accordingly.
14. You are allowed to decide some classes on basis of your knowledge gaining through your work with the related problem domain. In left textbox of Table #9 you just write the class name. Click 'Add' to take that class in our solution.
15. Click 'Top Level Classes' button to get all the top level classes to be generated for our problem statement.
16. Table #10 shown you all the domain classes for the given problem statement.

CONCLUSION

In this experiment, we successfully learned how to identify potential classes from a given problem statement using the noun identification method. We understood that nouns in the problem statement can represent potential objects or classes, while their attributes and methods help define the characteristics and behavior of those classes. We also learned to select only the relevant classes that fall within the scope of the system. This experiment helped us understand the basic concepts of object-oriented analysis and Domain Object Modeling.

EXP: 6	**AIM:- In this experiment, we will learn about the different components of activity diagram and state chart diagram and how these can be used to represent the dynamic nature of an information system.**

INTRODUCTION

Capturing the dynamic view of a system is very important for a developer to develop the logic for a system. State chart diagrams and activity diagrams are two popular UML diagram to visualize the dynamic behavior of an information system.

PRETEST

A state can contain one or more sub-state(s) within it
- (●) a: True
- ( ) b: False

What does forking of several activities from a synchronization point
- ( ) a: All those activities should get executed one after another
- (●) b: The activities can be performed in parallel
- ( ) c: One or more activities could be skipped

A decision point in an activity diagram is a control wher
- ( ) a: Multiple parallel activities merge
- ( ) b: Decision is taken whether a transition should happen
- (●) c: A condition is checked and decided which activity should be performed next
- ( ) d: There is no such control

Submit Quiz

Score: 3 out of 3

POSTTEST

What does an entry action of a state indicate?
- (●) a: Action performed after the system moves into the given state
- ( ) b: Action performed before system moves into the given state
- ( ) c: An optional action performed when system moves into the given state
- ( ) d: None of the above

What does the guard condition depicted over the transition betwe
- (●) a: A condition that must be true for the transition to happen
- ( ) b: A condition that must be false for the transition to occur
- ( ) c: An indicator that this transition should not happen
- ( ) d: An event that might happen as result of the transition

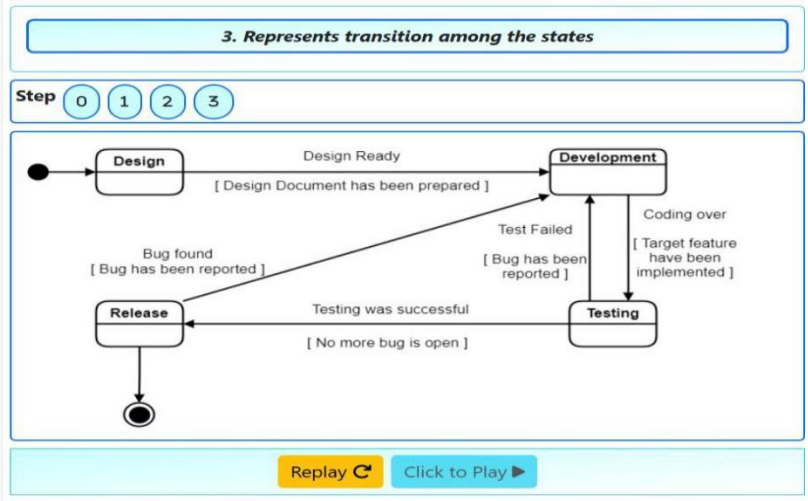
Submit Quiz

Score: 2 out of 2

SIMULATION

Drawing statechart diagram from a problem statement

Consider the development of a medium-scale software in, say, C++. In a software development life cycle typically four phased could be observed. It begins with planning and design, after which the development work starts. Once the required features have been implemented, the software is tested. If testing is successful, the software is released. If there is even a single failure, that is to be fixed, and tested again. However, certain bugs might become apparent only after someone uses the software in real scenario. As such, the software again enters into development phase, followed by testing, and then another release.



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

CONCLUSION

In this experiment, we successfully learned about the different components of Activity Diagrams and State Chart Diagrams and their role in representing the dynamic behaviour of an information system. We understood concepts such as activities, states, transitions, decision points, fork, join, entry actions, and guard conditions. These diagrams help in clearly representing the flow of activities and changes in the state of a system, making it easier to understand, analyze, and design the dynamic behaviour of software systems.

EXP: 7	**AIM:- In this Experiment, we will learn about the representation of class diagram and sequence diagram. We also learn about different relationships that exist among the classes, in a system.** **From the experiment of sequence diagram, we will learn about different types of messages passing in between the objects and time ordering of those messages, in a system.**

INTRODUCTION

Classes are the structural units in object oriented system design approach, so it is essential to know all the relationships that exist between the classes, in a system. All objects in a system are also interacting to each other by means of passing messages from one object to another. Sequence diagram shows these interactions with time ordering of the messages.

PRETEST

A class is a
- (●) a: Blueprint
- ( ) b: Specific instance of an object
- ( ) c: Category of user requirement
- ( ) d: None of the above

In class diagrams, a class is represented with a
- (●) a: Rectangle
- ( ) b: Human stick figure
- ( ) c: Ellipse
- ( ) d: Diamond

From a class diagram it is evident that
- ( ) a: All classes work in isolation
- ( ) b: Each class is related with every other class
- (●) c: Most of the classes are related
- ( ) d: Class diagram show object interactions

What does a sequence diagram represent?
- ( ) a: Workflow in the system
- ( ) b: How classes are related to each other
- ( ) c: Sequence of events flow to achieve a target
- (●) d: Sequence of activities to be performed before moving to next state

Submit Quiz

POSTTEST

Inheritance among classes are represented by a
- ( ) a: Solid line from the extending to the extended class
- (●) b: Line with an unfilled arrow head from the extending to the extended class
- ( ) c: Line with a filled diamond from the extending to the extended class
- ( ) d: Dotted line with extend stereotype from the extending to the extended class

Private members (or methods) in a class are indicated with a
- ( ) a: Hash (#) sign
- (●) b: Minus (-) sign
- ( ) c: Plus (+) sign
- ( ) d: Tilde (~) sign

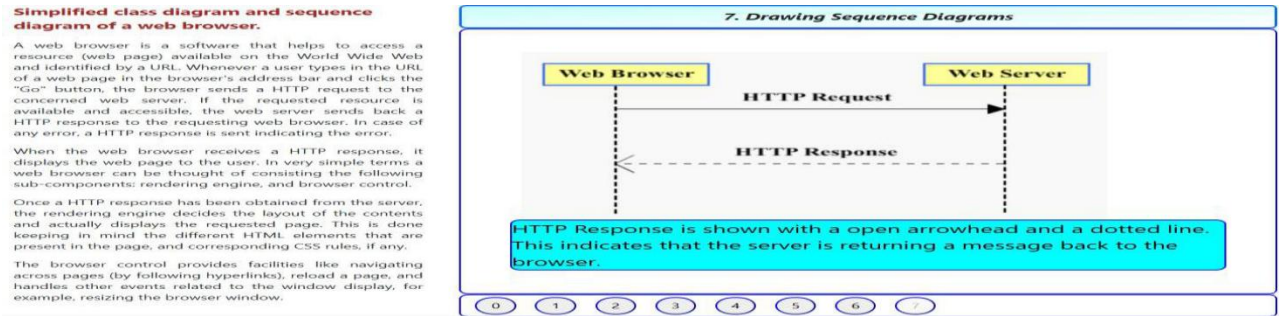
In UML 2.0 a synchronous message is represented with a
- (●) a: Solid arrow with filled arrowhead
- ( ) b: Solid arrow with empty arrowhead
- ( ) c: Dotted arrow with filled arrowhead
- ( ) d: Dotted arrow with empty arrowhead

An object can send a synchronous message and multiple asynchronous mess
- (●) a: True
- ( ) b: False

Submit Quiz

Score: 4 out of 4

SIMULATION



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

Steps for Drawing Class Diagram

1. Read the problem statement carefully and choose the work space specified for class diagram.
2. Identify a class, and write its name in the left textbox of 'Table #1'. Click the adjacent 'Add' button to add this class. Repeat this for all the possible classes.
3. You may add attributes to a class from 'Table #2'. Select a class from the left drop down list of 'Table #2'. Identify an attribute, and write its name in the next textbox of 'Table #2' and select the attribute type from the next drop down list. Click the adjacent 'Add' button to add this attribute. Repeat this for all attributes of all classes.
4. You may add operations to a class from 'Table #3'. Select a class from the left drop down list of 'Table #3'. Identify an operation, and write its name in the next textbox of 'Table #3' and select its return type from the next drop down list. Click the adjacent 'Add' button to add this operation. Repeat this for all operations of all classes.
5. The list of attributes and operations along with their respective classes are shown in 'Table #6'.If you want to delete any class, attribute or operation, you may delete it from here.
6. 'Table #4' lets you define inheritance relationship between any two classes. Select one sub-class from the left drop down list, select its super-class from the next dropdown list. Click the adjacent 'Add' button to add this inheritance relation. Repeat this for all inheritance relationships that you find for the system.
7. If you want to delete any inheritance relation, go to the 'Table #7', remove the sub-class or super-class.
8. 'Table #5' lets you define association relationships between any two classes. Select the first class name from the first dropdown list, the second from the fifth (right most) drop down list. Select one association type from the second dropdown list. If you want to put any name to the association, write the text for the name to the second textbox. If you wish to give role name to any one or both end of association then you write the name in first or third or in both text boxes. Click on the 'Add' button at the right side to add this association. Repeat this for all the possible associations.
9. Associations so defined will be displayed in the 'Table #8'. Here, you have the option to remove a wrongly defined associations.
10. Click on the 'Draw' button to draw the class diagram.

11. To view a sample solution for the exercise, click on the 'Submit' button, and then on the 'View Solution' button.

CONCLUSION

In this experiment, we successfully learned the basic concepts and notations used in UML class and sequence diagrams. We understood how inheritance is represented using a hollow arrowhead, how access modifiers such as private, public, and protected are represented, and how synchronous messages are shown in sequence diagrams. This experiment helped us understand how UML diagrams represent relationships, visibility, and communication between classes and objects in a software system.

EXP: 8	**AIM:- In this experiment we will learn how external entities and functionalities of any system can be captured; and flow of data across the system.**

INTRODUCTION

Information Systems (IS) help in managing and updating the vast business-related information. Before designing such an IS, it is helpful to identify the various stakeholders, and the information that they would be exchanging with the system. An IS, however, is a large software comprised of several modules, which, in turn, share the process the available data. These data are often stored in databases for further references. A Data Flow Diagram (DFD) is used to pictorially represent the functionalities of the ISs by focusing on the sources and
destinations of the data flowing in the system.

PRETEST

A DFD represents
- a: Flow of control
- b: Flow of data
- c: Both of the above
- d: Neither one

Which is not a component of a DFD?
- a: Data flow
- b: Decision
- c: Process
- d: Data store

Submit Quiz
Score: 2 out of 2

POSTTEST

How many processes are present in level-0 or context diagram?
- a: 0
- b: 1
- c: 2
- d: 3

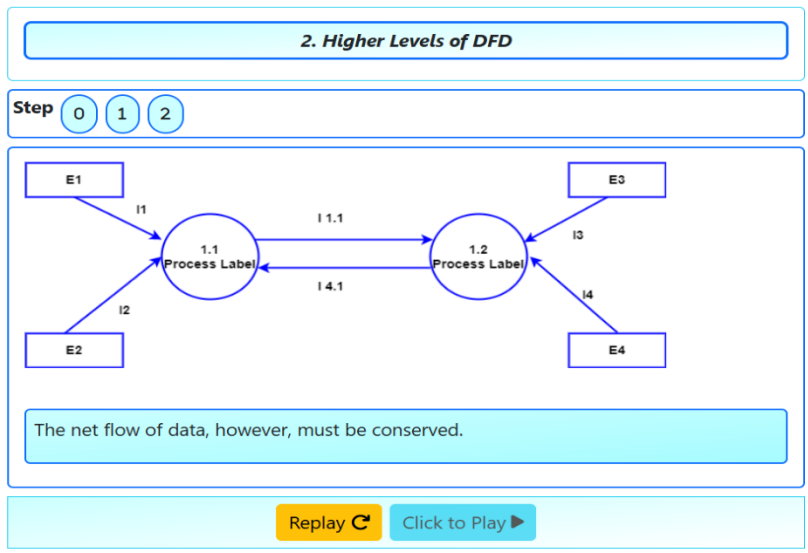
External entities can appear in a DFD
- a: At any level
- b: Only at level-0
- c: Only at level-1
- d: Either at level-0 or at level-1

Data flow in a DFD is not possible in between
- a: Two processes
- b: Data store and process
- c: External entity and data store
- d: Process and external entity

Submit Quiz
Score: 3 out of 3

SIMULATION

A DFD provides an easy mechanism to identify the flow of data in any information system. The adjoining simulation illustrates the generic steps to draw a DFD, and some thumb rules to keep in mind while drawing it.



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. Identify an external entity, and write it in the left textbox of 'Table #1'. Click the adjacent 'Add' button to add this external entity. Repeat this for all the possible external entity.
2. Identify a process, and write its name in the left textbox of 'Table #2' and put the level of that process in the next textbox of 'Table #2'. Click the adjacent 'Add' button to add this process. Repeat this for all the possible processes.
3. Identify a data store, and write its name in the left textbox of 'Table #3'. Click the adjacent 'Add' button to add this data store. Repeat this for all the possible data stores.
4. If the respective table flashes with green, then it means your entry is taken to the system and you can check it in 'Table #5'. If it flashes with red, then it means your entry is not taken to the system and you are requested to follow the messages generated by the system in each time.
5. If you want to delete any external entity, process or data store, remove it from 'Table #5'.
6. 'Table #4' lets you define the data-flows in between the components in a data flow diagram. Select the type of components from the first dropdown list, the second type from the third drop down list (4th column in the table). Select one component from the second and the other from the fourth (5th column in the table) dropdown lists which are participating in a data-flow. You need to put the label of the data-flow and write the text for the label to the adjacent textbox. Click on the 'Add' button at the right side to add this data flow. Repeat this for all the possible entries.
7. Data-flows so defined will be displayed in the 'Table #6'. Here, you have the option to remove a wrongly defined flow.
8. Click on the 'Draw' button to draw the Data Flow diagram. You can click on this button at any time to reflect the changes that you have made to the external entities, processes, data stores or data-flows. The Data Flow diagram will be displayed at the bottom of the page.
9. To view a sample solution for the exercise, click on the 'Submit' button, and then on the 'View Solution' button.

CONCLUSION

In this experiment, we successfully learned how to identify and represent external entities, processes, data stores, and data flows of a system using a Data Flow Diagram (DFD). We understood how data moves between different components and how the functionalities of a system can be represented at different levels. This experiment helped us understand the flow of information within a system and provided a clear graphical representation for analyzing and documenting system requirements.

EXP: 9	**AIM:- In this experiment, we will learn about basic blocks and how to draw a CFG using them. We would look into paths and linearly independent paths in context of a CFG. Finally, we would learn about McCabe's cyclomatic complexity, and classify a given program based on that.**

INTRODUCTION

A visual representation of flow of control within a program may help the developer to perform static analysis of his code. One could break down his program into multiple basic blocks, and connect them with directed edges to draw a Control Flow Graph (CFG). A CFG of a program helps in identifying how complex a program is. It also helps to estimate the maximum number of test cases one might require to test the code.

PRETEST

What does CFG of a program describe?

- a: Sequence of function calls
- b: Sequence of statements executed
- c: Contents of the stack
- d: There's nothing called CFG!

Submit Quiz

Score: 1 out of 1

POSTTEST

A set of paths are said to be linearly independent if

- a: Each of them is distinct
- b: No paths have a common node
- c: No two paths have a common node
- d: All the paths are pairwise distinct

According to McCabe's Cyclomatic complexity, V(G) = E - N + 2. Here, N is

- a: No. of statements in the program
- b: No. of unique operators used
- c: No. of nodes in the CFG
- d: No. of edges in the CFG

Submit Quiz

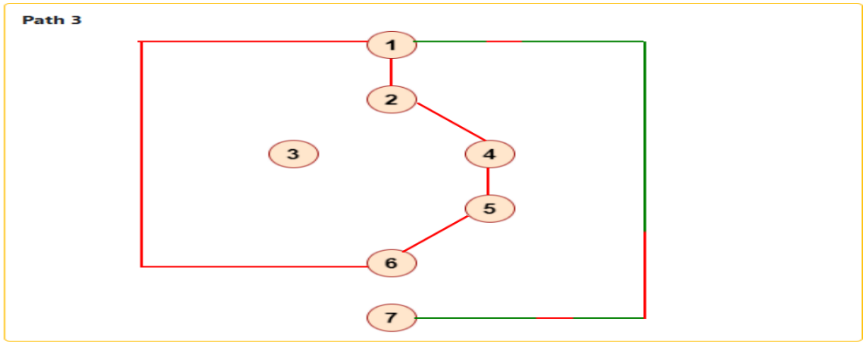
Score: 2 out of 2

SIMULATION

Code for GCD computation by Euclid's method

```
while (x != y) {
    if (x > y)
      x = x - y;
    else
      y = y - x;
}
return x;
```

Path 3



Determining McCabe's Cyclomatic Complexity

Method #1

N = No. of nodes = 7
E = No. of edges = 8
V(G) = E - N + 2 = 8 - 7 + 2 = 3

Method #2

V(G) = Total no. of non overlapping areas + 1 = 2 + 1 = 3

Method #3

V(G) = Total no. of decision statements and loops + 1 = 1 + 1 + 1 = 3

PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

Generating Control Flow Graph with gcc

1. As an example, consider the following simple C program:

```
#include <stdio.h>

int
main(int argc, char **argv)
{
        int n = 10;
        printf("Value of n: %d\n", n);

        return 0;
}
```

Figure-01

2. The basic blocks of the above program.

;; Function main (main)

Merging blocks 2 and 3

```
main (int argc, char * * argv)
{
  int n;
  int D.1705;
  const char * restrict D.1704;

<bb 2>:
  n = 10;
  D.1704 = (const char * restrict) "Value of n: %d\n";
  printf (D.1704, n);
  D.1705 = 0;
  return D.1705;

}
```

Figure-02

3. The following image shows a graphical representation of the above:.

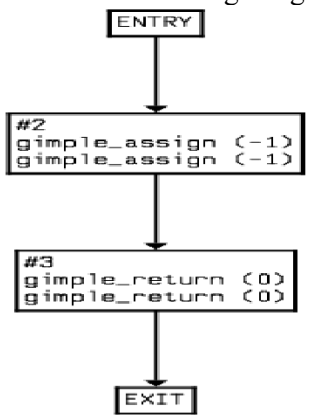


Figure-03

Following are the steps to work on the exercises for this experiment:

- The basic blocks is displayed on clicking the "Show Program Representation" button.
- Next click on the "Show Contro FLow Graph" button to display a graphical representation of the above program.
- To determine the McCabe's Cyclomatic Complexity, enter numeric values in the below fields.
- To verify your solution, click on the "Click to view solution" button below the problem statement. This would move you to the bottom of the page. Next click on the View Solution" button to display the solution.

CONCLUSION

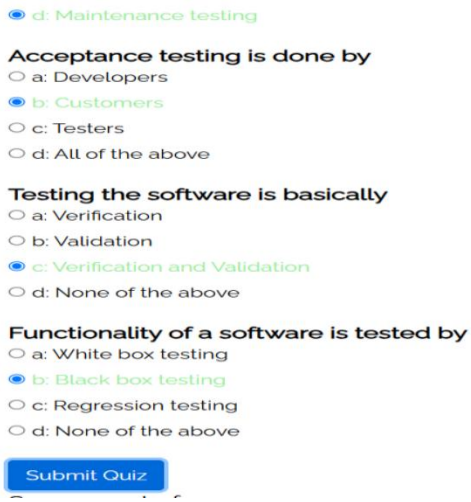
In this experiment, we successfully learned about the basic blocks used to construct a Control Flow Graph (CFG) and understood how to represent the flow of control in a program. We also learned how to identify paths and linearly independent paths in a CFG. Furthermore, we understood McCabe's Cyclomatic Complexity and how it can be used to measure the complexity of a program and determine the number of independent paths required for testing. This experiment helped us understand the importance of control flow analysis in software testing and quality assurance.

EXP: 10	AIM: In this experiment we discuss in brief about different types of testing, and provide mechanisms to have hands-on experience on unit testing.

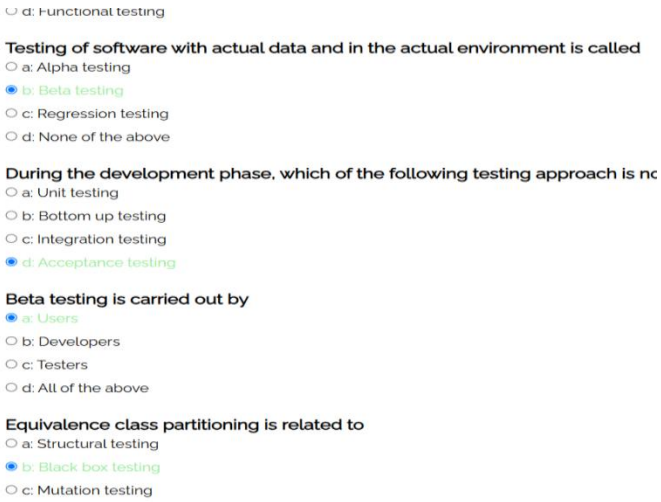
INTRODUCTION

Development of a new software, like any other product, remains incomplete until it subjected to exhaustive tests. The primary objective of testing is not to verify that all desired features have been implemented correctly. However, it also includes verification of the software behavior in case of "bad inputs".

PRETEST



POSTTEST



SIMULATION

Exhaustive testing of a software is required to determine it is working as per expectations and requirements. Developers often do not have enough time (or at times interest) to test their codes thoroughly. To handle such scenarios, most projects usually have a dedicated testing team. However, unit testing, at least, is done by the developers.

Irrespective of who performs testing, or what is being tested, testing usually involve some generic steps. In this simulation we provide a broad overview of the testing process.



PROCEDURE

Steps for conducting the experiment

General Instructions

Follow are the steps to be followed in general to perform the experiments in Software Engineering Virtual Lab.

1. Read the theory about the experiment
2. View the simulation provided for a chosen, related problem
3. Take the self evaluation to judge your understanding (optional, but recommended)
4. Solve the given list of exercises

Experiment Specific Instructions

Following are the instructions specifically for this experiment:

1. Type in the above code (in JavaScript) in the text area below the header "Code"
2. Click on the "Create Test Suite" button just below the code area.
3. Add a title and summary for the test suite to be created. When done, click on the "Add" button. If this test suite is not supposed to be added, click on the "Cancel" link.
4. After clicking on the "Add" button from the previous step, a dialog will display the new test suite. Every test suite is identified with a unique ID: an (auto-incrementing) integer prefixed with "TS". The first test suite will have the ID "TS0 ", and so on.
5. An already added test suite could be removed by clicking on the "Remove" button.
6. Once a test suite has been created, click on the "Add test cases" button to add the test cases individually.
7. After clicking on the "Add test cases" button a spreadsheet-type dialog will appear just below the button.
8. The number of columns in the spreadsheet are fixed (seven). The number of initial rows are 5.
9. NOTE: Summary and Script of a test case is mandatory for every test case in the test suite. If a particular test case has no summary, the concerned test case, and all other subsequent test cases, would be ignored!
10. Enter Expected value of the test case. Click on the "Execute test suite" button to execute the test cases for the concerned test suite. Please note that this may not execute all the test cases in the test suite if the constraints as mentioned in the previous step are not met. Also, any test case set for Manual testing would be skipped.
11. Results of execution of the test suite would appear just below the "Execute test suite" button. If all the test cases of the test suite have passed (i.e. values under Expected Output column is same as under Actual Output column for each row), the background colour of the results dialog would be green. Otherwise, it would be red.

CONCLUSION

In this experiment, we successfully learned about different types of software testing and understood their purpose in ensuring software quality and reliability. We also gained hands-on experience with unit testing and learned how individual components or modules of a software system can be tested independently to identify errors and defects. This experiment helped us understand the importance of systematic testing in developing reliable, functional, and high-quality software.